

浙江大学实验报告

专业：微电子科学与工程
姓名：赵楼晗
学号：3240104430
日期：2026 年 6 月 3 日
地点：紫金港东 4-223

课程名称：数字系统设计实验
实验名称：音乐播放实验

指导老师：屈民军、唐奕
实验类型：FPGA 应用实验

成绩：
同组学生姓名：

一、实验目的和要求

- (1) 掌握音符产生的方法，了解 DDS 技术的应用。
- (2) 了解按键处理的方法，掌握开关防颤动电路的设计。
- (3) 了解音频编解码的应用。
- (4) 掌握系统“自顶而下”的数字系统设计方法。

实验要求：

设计一个音乐播放器，满足以下条件：

- (1) 可以播放四首乐曲，设置 play/pause_button、next_button、reset 三个按键。按 play/pause_button 键，音乐在播放和暂停之间切换；按 next_button 键播放下一首乐曲。
- (2) LED0 指示播放情况（播放时点亮）、LED2 和 LED 3 指示当前乐曲序号。

二、实验内容和原理

2.1 直接数字频率合成技术 (DDS)

DDS 技术通过 ROM 存储正弦样品信号，从而实现在 FPGA 中快速地根据频率控制字 K 输出对应频率的正弦信号。DDS 的基本原理框图如图1所示，由相位累加器、正弦查询表 (Sine ROM)、DAC 转换器和低通滤波器组成。

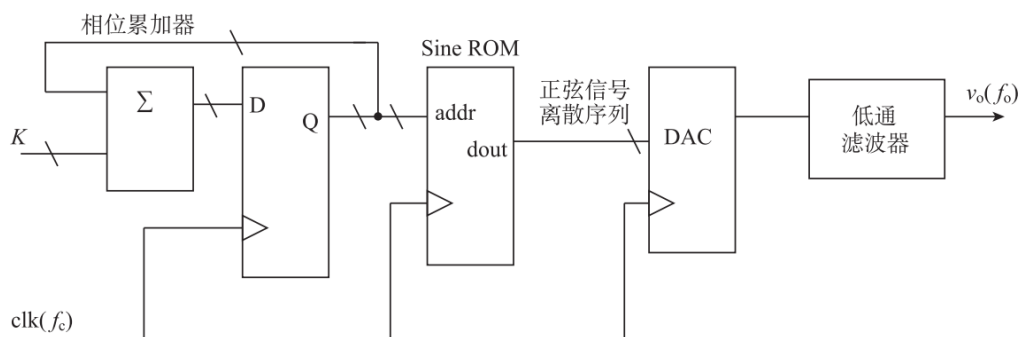


图 1: DDS 原理框图

在图1中，Sine ROM 中存放一个完整的正弦信号样品，其正弦信号样品与实际的正弦信号根据下式的映射关系构成：

$$S(i) = (2^{n-1} - 1) \times \sin\left(\frac{2\pi i}{2^m}\right), \quad i = 0, 1, 2, \dots, 2^m - 1$$

式中， m 为 Sine ROM 地址线位数； n 为 ROM 的数据线宽度； $S(i)$ 的数据形式为补码。

f_c 为取样时钟 clk 的频率， K 为相位增量（也称为频率控制字），输出正弦信号的频率 f_o 由 f_c 和 K 共同决定，即

$$f_o = \frac{K \times f_c}{2^m}$$

由上式可看出，正弦信号的频率 f_o 与相位增量 K 呈正比关系。注意： m 为图 6.12 中相位累加器的位数。为了得到更准确的正弦信号频率，相位累加器位数会增加 p 位小数。也就是说，相位累加器的位数是由 m 位整数和 p 位小数组成的。相位累加器的高 m 位整数部分作为 Sine ROM 的地址。

因为 DDS 遵循奈奎斯特（Nyquist）采样定律，即最高的输出频率是时钟频率的一半，即 $f_o \leq f_c/2$ 。实际中 DDS 的最高输出频率由允许输出的杂散电平决定，一般取值为 $f_o \leq 40\% \times f_c$ ，因此 K 的最大值一般为 $40\% \times 2^{m-1}$ 。

由于实际上正弦波具有中心对称的特点，我们只需知道 $\frac{1}{4}$ 周期的值就可以复原出整个周期内的值，因此我们在 ROM 中仅需存储一个正弦波 $\frac{1}{4}$ 周期内的采样值即可，这大大减少了 ROM 所需的存储空间；按照这种思路进行设计结构优化后的 DDS 模块如图 2：

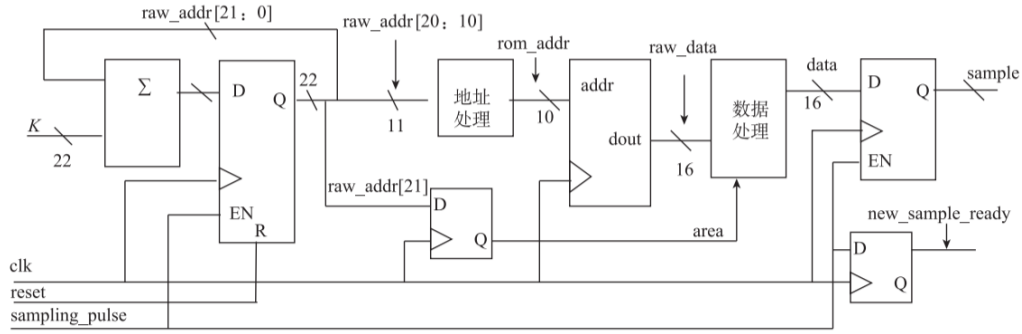


图 2: 优化后的 DDS 原理框图

2.2 音符产生原理

有了 DDS 技术，我们可以方便地产生正弦波，将对应频率与持续时长的正弦波依次通入扬声器中，便可以得到不同音高和节拍的音符。

实验中，用数组 {note,duration} 表示音符，note、duration 均为 6 位二进制数。note 为音符标记，其值含义如表 1 所示；duration 表示“音的长短”，其单位为 $\frac{1}{48}s$ 。实验用 $128 \times 12\text{bits}$ 存储器 (song_rom) 存放乐曲，每首乐曲最长由 32 个音符组成，可存放四首乐曲。

音频编解码器的取样频率为 48kHz，采用 $m = 12$ 的 Sine ROM。若已知音符频率 f 就可根据下式求相位增量 k ：

$$k = \frac{f(\text{Hz})}{11.7}$$

表 1 中的 k 就是根据上式计算出来的。为了更准确地得到音符频率，用 20 位二进制数表示 k ，其中高 10 位为二进制整数，低 10 位为二进制小数。

由于音符个数有限，采用查找表方法实现式 (6.29) 所示的除法运算较为方便，因此实验中还需要一个 Frequency ROM 来存放 k ，Frequency ROM 的地址与音符标记 (note) 相对应，这样，Frequency ROM 的大小为 $64 \times 20\text{bits}$ 。表 6.10 列出最常用音符对应的 k 值，表中 note 这一列可作为 Frequency ROM 的地址，而 k 为 Frequency ROM 的数据。

表 1: 音符频率对照表

分组	音符	频率/Hz	Note	k	备注
低音	1	262	16	{10'd22, 10'd365}	2C
	1#	277	17	{10'd23, 10'd652}	2C#Db
	2	294	18	{10'd25, 10'd90}	2D
	2#	311	19	{10'd26, 10'd551}	2D#Eb
	3	330	20	{10'd28, 10'd163}	2E
	4	349	21	{10'd29, 10'd800}	2F
	4#	370	22	{10'd31, 10'd587}	2F#Gb
	5	392	23	{10'd33, 10'd461}	2G
	5#	415	24	{10'd35, 10'd423}	2G#Ab
	6	440	25	{10'd37, 10'd559}	2A
中音	6#	466	26	{10'd39, 10'd783}	A#Bb
	7	495	27	{10'd42, 10'd245}	2B
	1	524	28	{10'd44, 10'd731}	3C
	1#	554	29	{10'd47, 10'd281}	3C#Db
	2	588	30	{10'd50, 10'd180}	3D
	2#	622	31	{10'd53, 10'd79}	3D#Eb
	3	660	32	{10'd56, 10'd327}	3E
	4	698	33	{10'd59, 10'd576}	3F
	4#	740	34	{10'd63, 10'd150}	3F#Gb
	5	784	35	{10'd66, 10'd922}	3G
中音	5#	830	36	{10'd70, 10'd846}	3G#Ab
	6	880	37	{10'd75, 10'd95}	4A
	6#	932	38	{10'd79, 10'd543}	4A#Bb
高音	7	990	39	{10'd84, 10'd491}	4B
	1	1048	40	{10'd89, 10'd439}	4C
	1#	1108	41	{10'd94, 10'd562}	4C#Db
	2	1176	42	{10'd100, 10'd360}	4D
	2#	1244	43	{10'd106, 10'd158}	4D#Eb
	3	1320	44	{10'd112, 10'd655}	4E
	4	1396	45	{10'd119, 10'd128}	4F
	4#	1480	46	{10'd126, 10'd300}	4F#Gb
	5	1568	47	{10'd133, 10'd821}	4G
	5#	1660	48	{10'd141, 10'd669}	4G#Ab
高音	6	1760	49	{10'd150, 10'd191}	5A
	6#	1864	50	{10'd159, 10'd62}	5A#Bb
	7	1980	51	{10'd168, 10'd983}	5B

2.3 开关防颤动电路原理

人们完成一次按键操作的指压力如图 3(b) 所示, 按键开关从最初按下到接触稳定要经过数毫秒的颤动, 按键松开时也有同样问题。因此, 按键被按下或释放时, 都有几毫秒的不稳定输出, 从逻辑电平角度来看不稳定输出期间, 其电平在“0”“1”之间无规则摆动。因此, 一次按键操作的输出如图 3(c) 所示。按键操作时间 t_a 因人而异, 一般开关 $t_a < 100\text{ms}$ 。开关防颤动电路的目的是: 按一次按键, 输出一个稳定脉冲, 即将开关的输出作为开关防颤动电路的输入, 而开关防颤动电路的输出为理想输出, 如图 3(d) 所示。

因此, 设计开关防颤动电路的关键就是避免在颤动期采样。颤动期时间长短与开关类型及按键指压力有关, 一般为 10ms 左右。据此可画出防颤动电路的原理框图, 如图 4 所示。开发板输入时钟 clk 的频率为 100MHz, 10^5 分频器

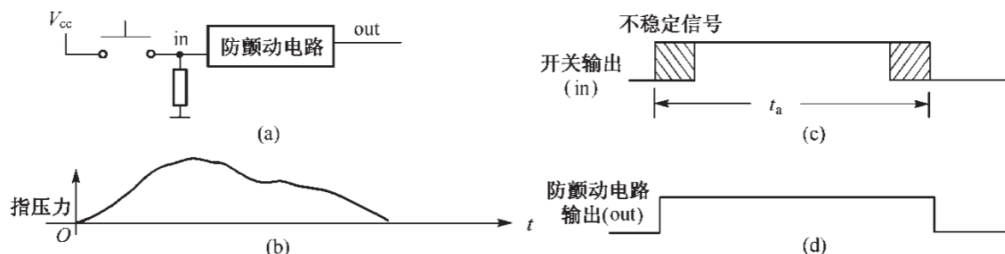


图 3: 按键开关颤动的产生

将产生周期为 1ms 的脉冲信号 pulse1kHz, pulse1kHz 的宽度为一个时钟 clk 周期, 即 10ns。脉冲信号 pulse1kHz 脉冲信号用作定时器的基准, 10ms 定时器是用来定时颤动期, 启动信号 timer_clr 由控制器提供, 定时结束信号 timer_done 反馈给控制器。

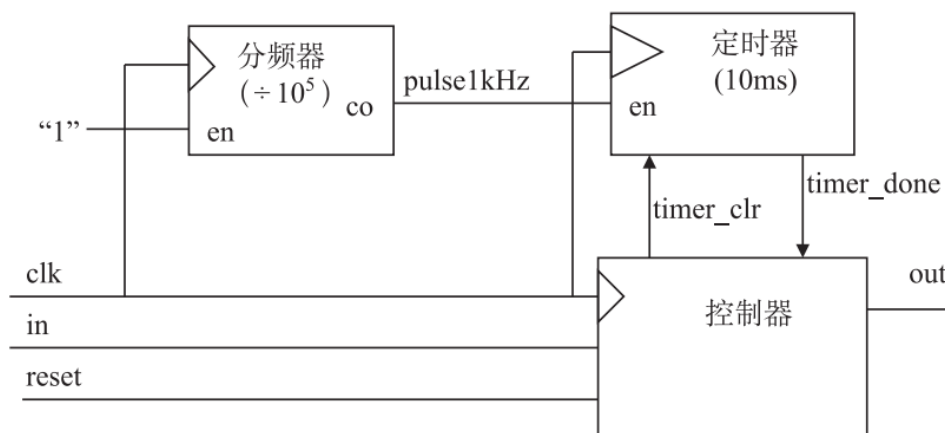


图 4: 防颤动电路原理框图

三、 主要仪器设备

- (1) 装有 Vivado、ModelSim SE 软件的计算机。
- (2) Nexys Video FPGA 开发板。
- (3) 头戴式耳机或其他音频输出设备。

四、 操作方法和实验步骤

4.1 前置子实验设计

4.1.1 DDS 模块的设计 (实验 15)

按照图2优化后的 DDS 原理框图, 对各个模块进行硬件语言描述; src 源码部分文件结构如下:

```
src/
├── dds.v ..... DDS 顶层模块
└── adder_22bit.v ..... 22 位加法器
```

| sine_rom.v 正弦查找表 ROM
 | full_adder.v 全加器
 | dds_tb.v DDS testbench 文件
 具体代码已包含在 solution 中提交，核心部分均有注释说明；下面对各模块功能与核心语句进行阐释：

sine_rom.v 为正弦查找表，其由老师提供的 SineROMGenerator.c 生成，其内部形如：

```

1 module sine_rom(
2   input      clk,
3   input wire[9:0] addr,
4   output reg[15:0] dout);
5   always @(posedge clk)
6     case(addr)
7       0:dout= 16'd0;
8       1:dout= 16'd50;
9       2:dout= 16'd101;
10      3:dout= 16'd151;
11      ...
12     1022:dout= 16'd32767;
13     1023:dout= 16'd32767;
14   endcase
15 endmodule

```

full_adder.v 为 1 位全加器模块，便于后续 22 位全加器模块的实例化调用；

adder_22bit.v 为 22 位全加器模块，对应 2 中的相位累加器，将 k 与 raw_addr 相加作为下一周期的 raw_addr。

dds.v 为 dds 模块的顶层，包含原理图中的 3 个 D 触发器与其他子模块，以及各模块间的接线等信息。其中 D 触发器通过 always 连续赋值语句结合非阻塞赋值实现，形如：

```

1   always @(posedge clk or posedge reset) begin
2     if (reset)
3       area <= 1'b0;
4     else if (sampling_pulse)
5       area <= raw_addr[21]; // raw_addr[21] 是最高位，决定正负半周
6   end

```

DDS 模块单独的仿真如图 5，在 ModelSim 中将输出信号转成模拟值显示可以显示出 DDS 模块产生的正弦波形；观察到不同的输入频率控制字 k 对应的不同频率的波形，并通过简单理论计算验证，基本符合公式 $f_o = \frac{k \times f_c}{2^m}$ ，可得出结论：DDS 模块设计基本符合要求，可进行下一步的设计。

4.1.2 开关防颤动电路设计（实验 11）

src 中文件结构如下：

```

src/
| button_system.v ..... 顶层模块（防颤 + 脉宽变换 + T 触发器）
| button_system_tb.v ..... 系统级仿真测试文件
| button_process_unit.v ..... 防颤电路核心模块
| button_process_unit_tb.v ..... 防颤电路仿真测试文件
| controller.v ..... 防颤状态机控制器
| divider.v ..... 105 分频器（产生 1ms 脉冲）

```

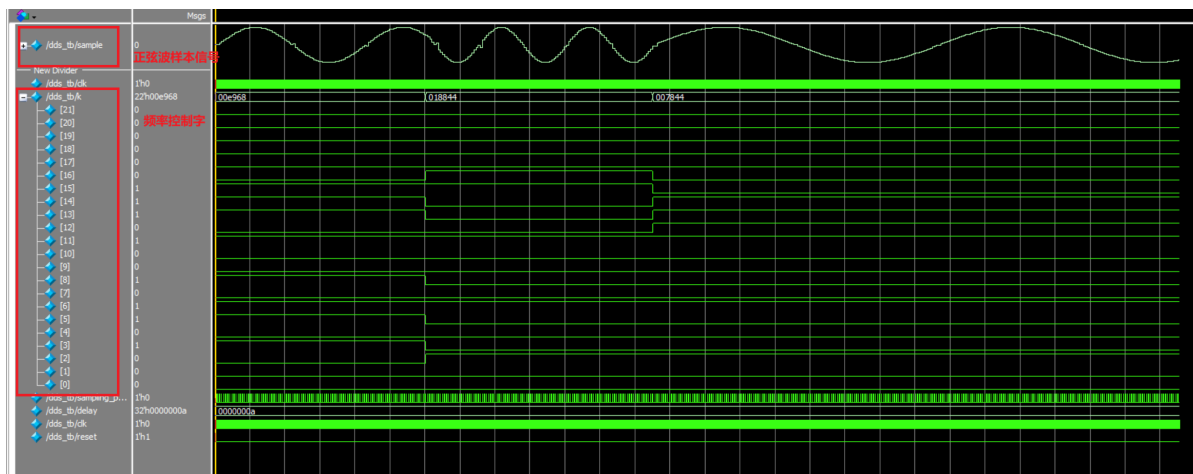


图 5: DDS 模块仿真波形图

timer.v 10ms 定时器
synchronizer.v 两级 D 触发器同步器
pulse_width_converter.v 边沿检测/脉宽变换
t_ff.v T 触发器 (LED 翻转测试)

button_process_unit.v 为核心的防颤动模块的顶层，整个模块由 controller.v 控制器、divider.v 分频器、timer.v 定时模块组成。其中控制器负责整个电路的状态机控制，分频器负责将 100 MHz 时钟分频为周期 1 ms 的单周期脉冲，定时器负责进行 10ms (颤动期) 的定时。

此外开关处理模块电路还包含了 synchronizer.v 同步器与 pulse_width_converter.v 脉宽变换模块，前者通过两级 D 触发器同步器消除异步按键输入的亚稳态，后者通过脉宽变化将去抖后的长电平转换为单周期脉冲。这两个模块在最终音频实验中也被调用了。

而测试电路本身的功能其实就是一个 T 触发器的功能：按钮按下一次 LED 的电平实现翻转，从而实现灯亮与灭的切换。

防颤动电路的仿真验证见图6，可见在一个颤动期内输出并没有随输入的颤动发生变化，因此认为该模块设计基本符合要求。

创建 vivado 工程并将代码下载入开发板中，按动按钮进行测试，发现 LED 灯瞬间亮起或熄灭，而不会出现颤动闪烁的情况，说明该模块功能正常，设计基本符合要求。

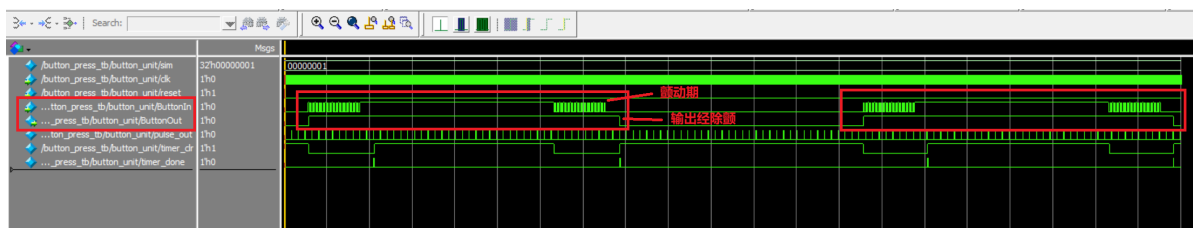


图 6: 防颤动电路模块仿真波形图

4.2 音频播放电路及其子模块设计

提交的 solution 中，src 源码部分的文件结构如下：

src/

MusicPlayerTop.v	FPGA 顶层模块
MusicPlayer.xdc	Vivado 管脚约束
mcu/	
mcu.v	主控制器
mcu_tb.v	主控制器 testbench 文件
music_player/	
music_player.v	音频播放器次顶层
music_player_tb.v	音频播放器 testbench 文件
note_player/	
note_player.v	音符播放顶层
note_player_controller.v	音符播放控制器
note_timer.v	节拍定时器
frequency_rom.v	音符与频率控制字对应表
note_player_tb.v	音符播放 testbench 文件
dds/	
dds.v	DDS 模块
adder_22bit.v	22 位加法器
sine_rom.v	正弦查找表 ROM
full_adder.v	全加器
dds_tb.v	DDS testbench 文件
song_reader/	
song_reader.v	乐曲读取顶层
song_reader_controller.v	乐曲读取控制器
song_addr_counter.v	5 位地址计数器
song_rom.v	乐曲存储 ROM
song_reader_tb.v	乐曲播放 testbench 文件
synchronizer/	
synchronizer.v	同步化电路
synchronizer_tb.v	同步器 testbench 文件
divider/	
divider.v	节拍分频器
divider_tb.v	分频器 testbench 文件
audiointerface/	
AudioInterface.v	音频编解码接口（端口声明）
AudioInterface.edf	音频编解码接口（网表）
button_process_unit/	
button_process_unit.v	按键处理模块（端口声明）
button_process_unit.edf	按键处理模块（网表）
my_button_process/	
button_system.v	自设计按键处理模块
:	（其余子模块文件）

在阅读工程文件中，发现该实验中老师已提供按键处理模块的 Blackbox，因此在工程中选用了老师提供的模块，自己写的模块仅作为参考一并入文件夹中。

接下来对于其中的重要子模块进行阐述。

4.2.1 音乐播放器

音乐播放器 (music_player) 次顶层又分为主控制器 (mcu)、单音符播放器 (note_player)、乐曲读取模块 (song_reader)、以及分频模块 (divider) 等，下面将分别阐述设计思路及核心代码。（更严谨的文件结构应该是上述模块的文件全都放在 music_player 子目录中，但是为了更加直观地展示文件结构和代码，我将上述模块放到了和 music_player 平级的目录下）

主控制器设计

主控制器为 4 状态的有限状态机，四个状态分别为：RESET, PAUSE, PLAY, NEXT，对应音乐播放器的实际四个工作状态。主控制器根据自身的状态发出相应的控制指令，控制音乐播放器播放与否，其算法流程图（ASM）见图7，可按照 ASM 描述主控制器模块的行为逻辑。

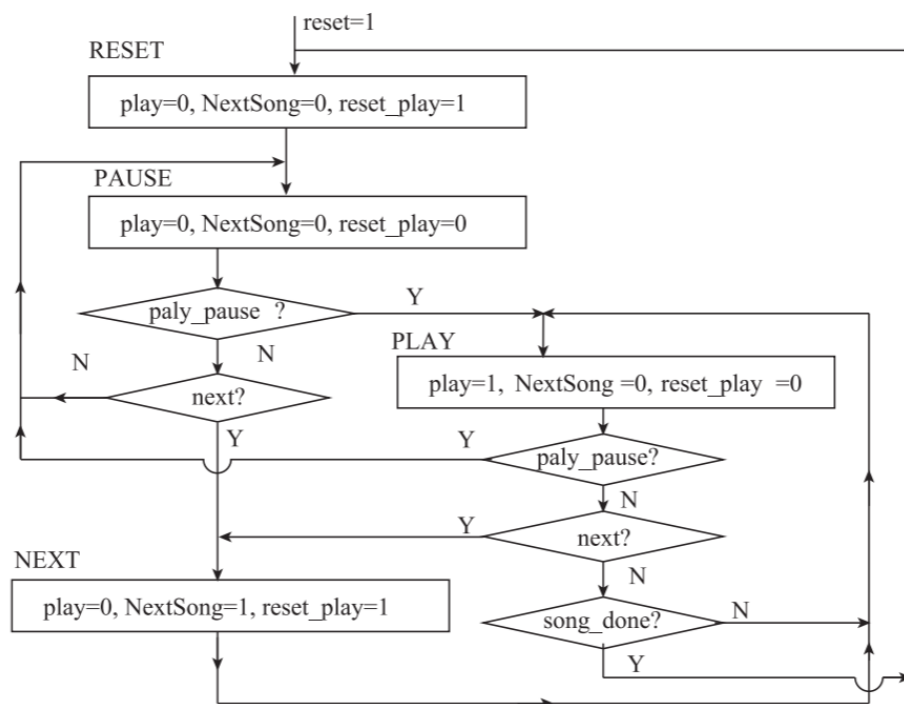


图 7: mcu 算法流程图

单音符播放器设计

单音符播放器是本次实验的核心，其根据频率对照表确定单个音符的频率控制字 k ，再利用 DDS 子模块产生对应频率的正弦波，实现单个音符的发声功能。

设计单音符播放器时同样遵循“控制器-各子模块”的结构，控制器负责整体的控制信号，其他子模块负责各种数据信号。根据教材中给出的模块接口描述、结构框图与控制器 ASM 描述，可完成单音符播放器模块的 Verilog 描述。

乐曲读取模块设计

乐曲读取模块的功能是：读取 song_rom.v 存储好的歌曲简谱，在合适的时候告诉单音符播放器应该播放哪个音；同时受控于主控制器，在暂停、播放、下一首歌等按键按下时执行对应操作。

song_rom.v 文件中定义了 $128 \times 12\text{bit}$ 的 ROM 存储器，其中 32 个存储单元为一首歌的内容，共有四首歌。文件中第四首歌所有音符和时长都设成了 0，需要我们自行选歌填入。此处我选择了贝多芬的钢琴名曲《致爱丽丝》(Für Elise) 作为自选歌曲，参考五线谱见文末图20，由于存储器大小限制只选前 32 个音符填入。

4.2.2 同步化电路

同步化电路用来进行音频编解码接口模块和其他模块的控制、应答信号的时钟同步化处理。

实验中音频编解码接口模块的输出信号 NewFrame 的脉冲宽度为一个 audio_clk 时钟周期，需通过同步化处理，产生与 sys_clk 同步且脉冲宽度为一个 sys_clk 时钟周期的信号 ready。根据教材中对于同步化电路的描述以及给出的参考电路框图（图8），可以完成同步化电路模块的 Verilog 描述。

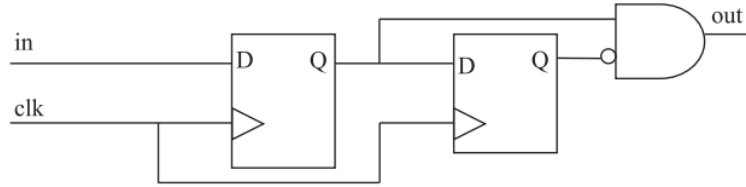


图 8: 同步化电路框图

4.2.3 节拍分频器

节拍分频器的作用为将 12.5MHz 的音频时钟作 1000 分频处理，产生用于提供音乐“节拍”的时钟信号 beat。

分频器的核心通过计数器实现，即一个时钟周期计一次数，当计数达到分频比后输出 beat 置一个时钟周期的高电平，其余时候置低电平。需要注意的是，分频器模块需设置仿真加速参数 sim，当启用仿真时（sim = 1）采用 64 分频比；模块内通过选择语句根据仿真参数选择对应的分频比和位宽：

```
1 module divider #(
2     parameter sim = 0          //默认不采用仿真加速
3 ) (
4     input wire clk,
5     input wire reset,
6     input wire sampling_pulse, // 48kHz 单周期脉冲
7     output reg beat            // 48Hz 单周期脉冲
8 );
9
10 // sim=0: 10位/1000分频, sim=1: 6位/64分频
11 localparam WIDTH = sim ? 6 : 10;
12 localparam MAX = sim ? 63 : 999;
13     . . .
14 endmodule
```

4.3 音频播放电路仿真验证

提交的 solution 中，sim 子文件夹对应各子模块与整体电路的仿真文件，其文件结构如下：

```
sim/
├── dds/.....DDS 模块仿真工程
├── divider/.....分频器仿真工程
├── mcu/.....主控制器仿真工程
├── note_player/.....音符播放器仿真工程
└── song_reader/.....乐曲读取仿真工程
```

—synchronizer/	同步器仿真工程
—music_player/	次顶层仿真工程

使用 ModelSim 对各子模块进行仿真验证功能，观察关键信号的时序波形，确认各模块功能描述正常；随后对整体次顶层进行仿真，观察时序波形，确认整体功能正常。

仿真波形结果将在“实验数据记录和处理”部分详细展示。

4.4 vivado 工程创建

经过仿真验证电路功能正常后，便可创建 vivado 工程将代码上传入 FPGA 中。vivado 工程文件位于上传的 solution/vivado 子目录下。

将 src 中除 testbench 外的所有文件加入 vivado 工程中，并加入老师提供的 xdc 引脚约束文件与音频编解码接口模块、按键处理模块的接口、网表文件（最终验收时使用了老师提供的按键处理模块而非自行编写的），并按要求配置基于 IP 的 DCM 时钟管理模块，设置 100MHz 的系统时钟和 12.5MHz 的音频时钟，根据网表生成图9的 RTL 级电路原理图。

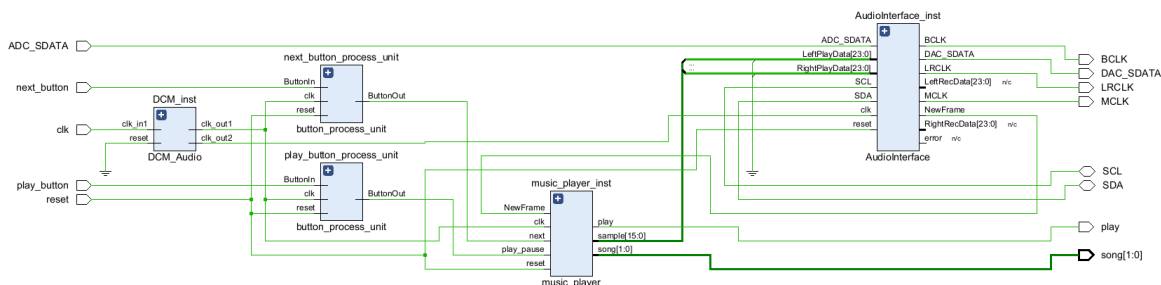


图 9: 电路原理图

依次进行综合（Sythesis）、实现（Implementation），再生成比特流，连接至 Nexys Video FPGA 并上传。由于引脚约束文件已经一并放入工程中，无需再额外配置引脚约束，只需检查无误即可。

最终音乐播放结果已在验收中展示，报告中不再赘述。

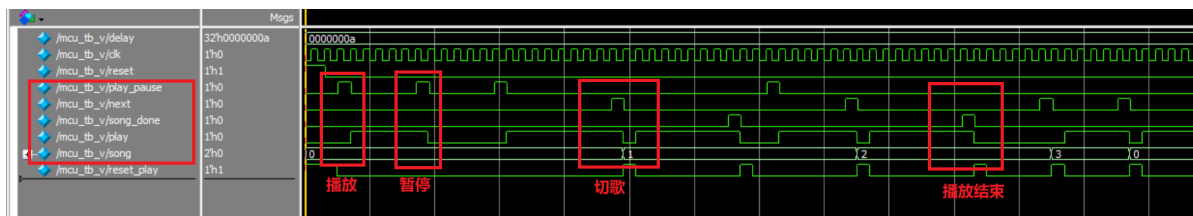
五、 实验数据记录和处理

本部分对仿真结果进行详细说明，其中 DDS 模块和防颤动电路模块的仿真波形已在子实验部分给出，此处阐述主实验中其余重要模块的仿真结果。

5.1 主控制器仿真波形

主控制的仿真波形见图10；可重点关注其中的 play_pause、song_done、next 等输入信号与 play、song 等输出信号。

观察到，当 play_pause 置一周期的高电平时，play 信号发生翻转，表示按下播放暂停按钮，音乐播放或暂停；当 next 信号置一周期高电平时，song 信号加一，表示播放下一首歌；当一首歌播放结束、song_done 出高电平时，play 变低电平，表示结束播放，进入暂停状态等待输入。因此可以得出结论，主控制设计基本符合要求。



5.2 音符播放器仿真波形

音符播放器的波形见图11；可重点关注其中的输入 dds_k、timer_done 和输出 sample 波形、note_done 等信号。

观察到，在一个音的持续时长内，输出信号 `sample` 为频率由 DDS 频率控制字控制的正弦波；当一个音结束后，即 `timer_done` 出高时，`dds_k` 读取下一个音的频率控制字，`sample` 信号的频率随之改变，同时 `note_done` 出一个周期的高电平表示刚刚结束一个音符的播放。

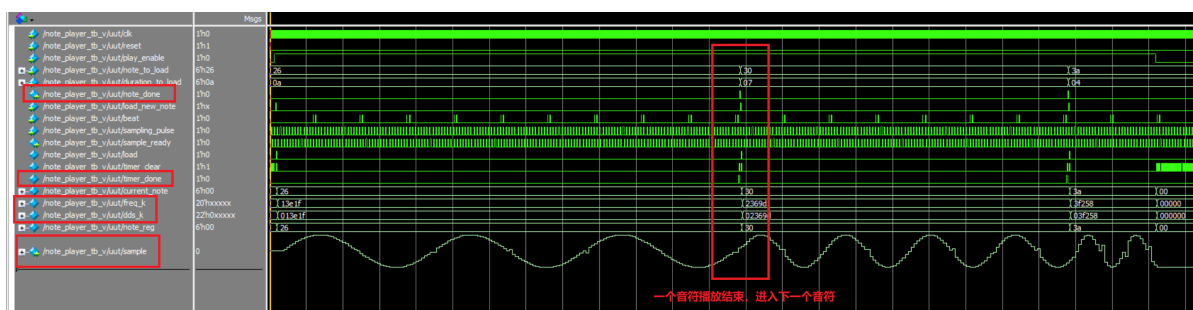


图 11: 音符播放器仿真波形图 1

放大时间轴可以更加清晰的看到正弦 sample 信号的频率转变：图12

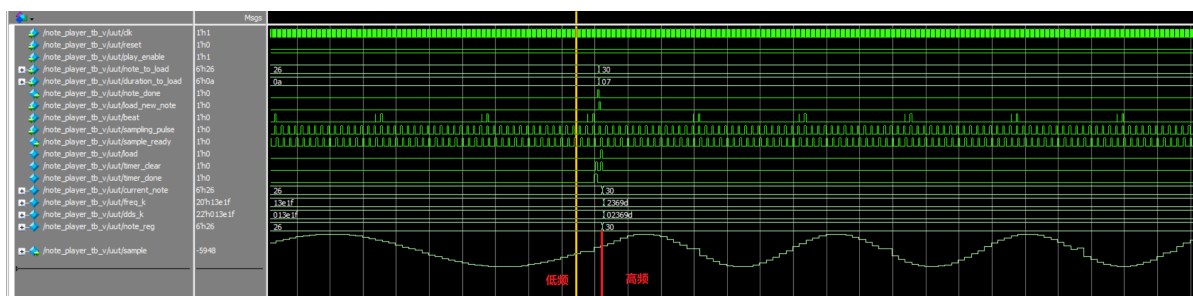


图 12: 音符播放器仿真波形图 2

5.3 乐曲读取仿真波形

乐曲读取模块的仿真波形见图13；可重点关注其中的输入 note_done、song_done 信号，以及输出 note、duration 等信号。

当音符播放器输入 `note_done` 信号表示上一个音播放结束后，乐曲读取模块首先更新下一个音的 ROM 地址 `addr`，再下一个周期同步更新音符的频率 `note` 以及持续时长 `duration`，同时将 `new_note` 信号置一周期的高电平；当输入 `song_done` 信号时，`duration_zero` 信号置高电平，表示当前乐曲播放结束。

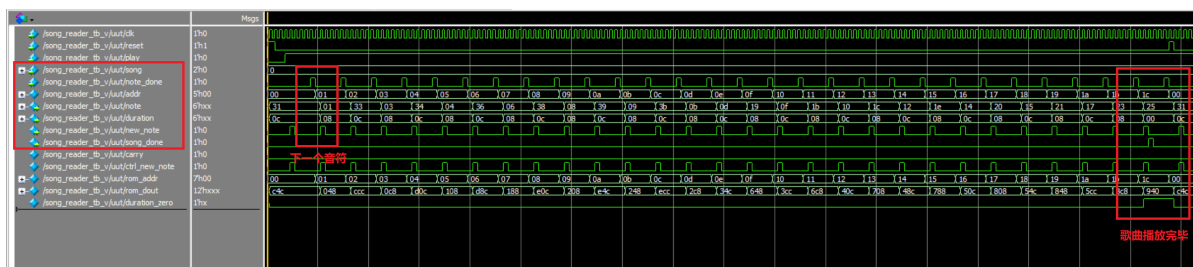


图 13: 音乐读取仿真波形图 1

放大时间轴，可以更加清晰地看到切换音符时几个信号变化的时序关系，和教材中给出的一致：图14

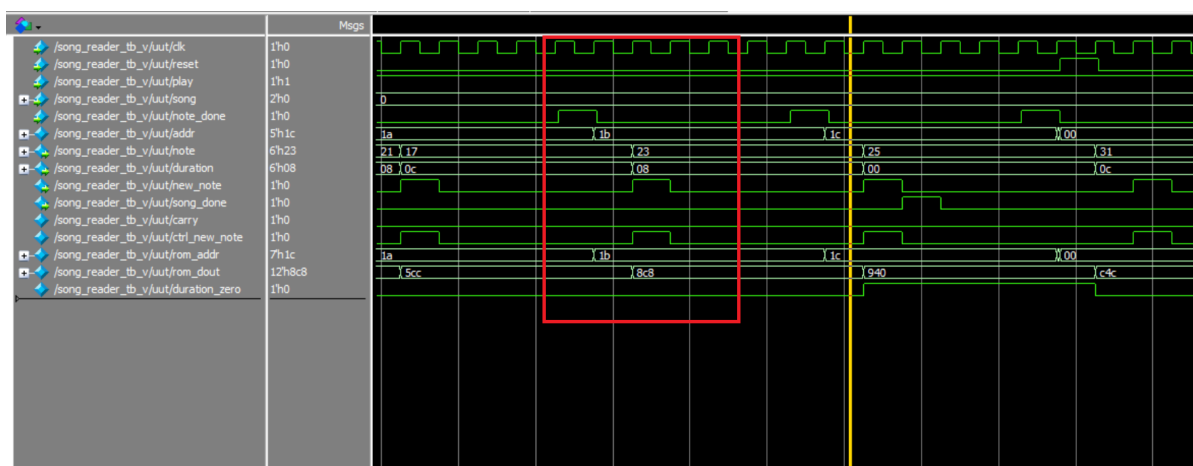


图 14: 音乐读取仿真波形图 2

5.4 分频器仿真波形

分频器模块仿真波形见图15；观察其中的输入 sample_pulse 脉冲信号与输出 beat 信号，经仔细验证确实为 sample_pulse 经过 64 个脉冲后 beat 输出一个脉冲，说明在仿真中 64 分频功能正确；该模块的 testbench 文件没有给出，为自行编写。

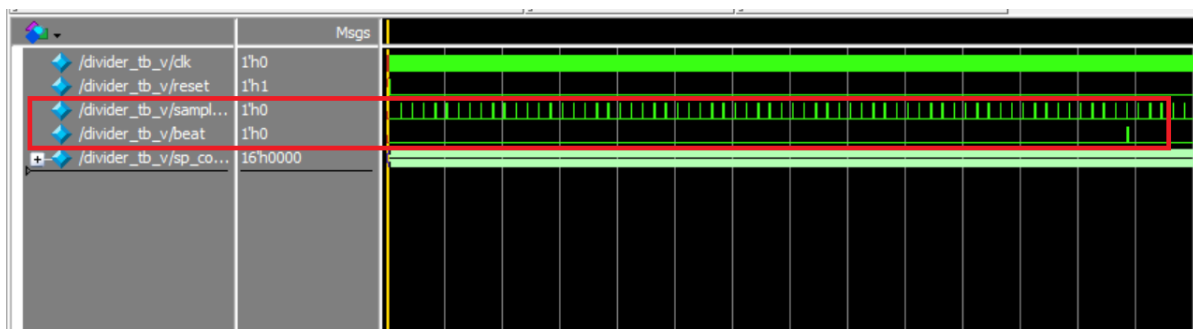


图 15: 分频器仿真波形图

5.5 同步化电路仿真波形

同步化电路仿真波形见图16；可重点关注其中的输入 NewFrame、两个不同频率的时钟信号与输出 ready 信号；观察到 NewFrame 出一个音频时钟周期（audio_clk）的高电平时，ready 信号输出一个系统时钟周期（audio_clk）

的高电平，完成了同步时钟的功能。

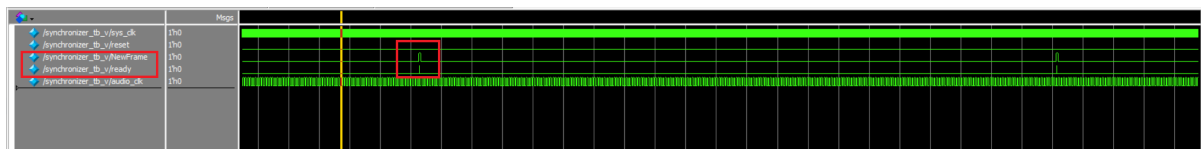


图 16: 同步化电路仿真波形图 1

图17放大查看了输出的高电平宽度，发现输出 ready 确实为一个系统时钟周期的宽度，说明基本功能正确。

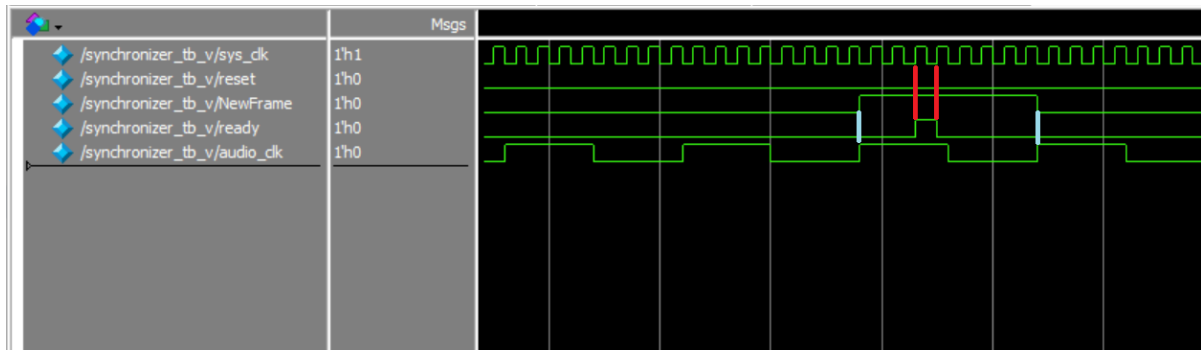


图 17: 同步化电路仿真波形图 2

5.6 次顶层仿真波形

观察到各模块仿真功能正常，接下来进行整体次顶层模块的仿真验证：图18、图19

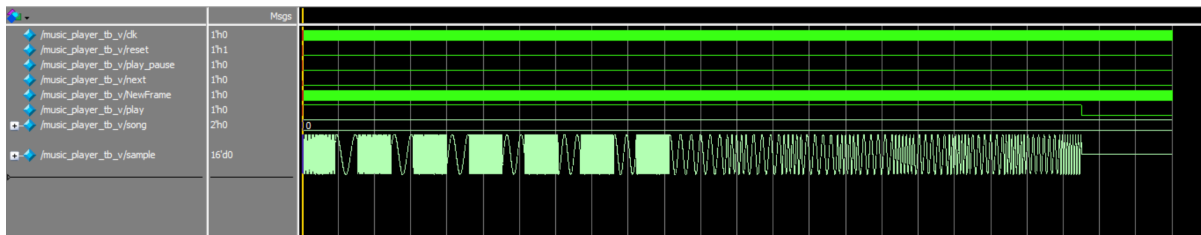


图 18: 音乐播放器电路仿真波形图 1

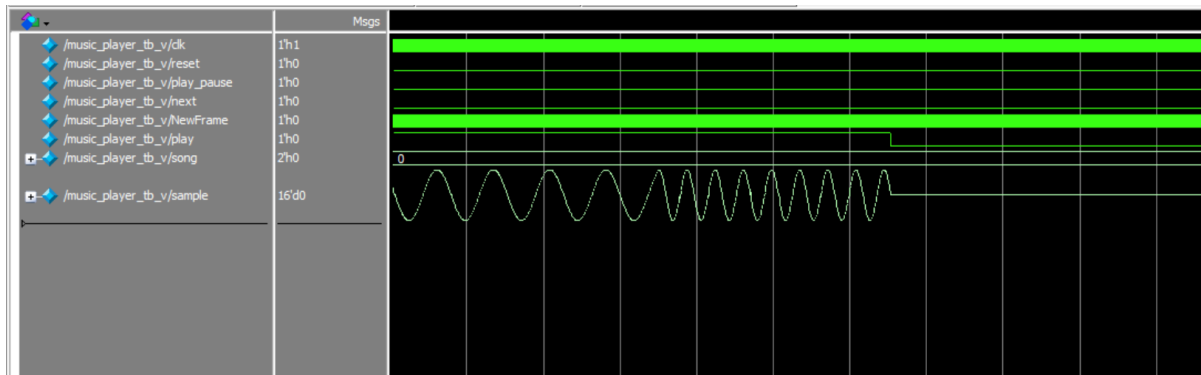


图 19: 音乐播放器电路仿真波形图 2

由于仿真的 testbench 文件中并未写入按下 play/pause 按钮与 next 按钮，因此该仿真的全部功能仅为：完整播放完第一首歌，并暂停播放。图18中的 sample 代表每一个播放的音，观察到其在不同频率间切换，对应歌曲中不同频率的音调；同时观察到图19中，歌曲播放完最后一个音后，play 信号置低电平，同时 sample 信号变为 0，说明歌曲播放结束、音乐播放器暂停播放。因此可以得出结论，该次顶层模块功能基本正常。

六、实验结果分析、总结与心得

观察分析各子模块与次顶层模块仿真的波形，其结果都符合设计的时序要求，也与教材中的示例波形基本一致；最后烧录入 FPGA 后音乐播放器的功能也基本正常，可完成暂停/继续、切歌、复位等基本功能，歌曲播放完成后不会自动播放下一首歌而是回到暂停状态；播放音频的频率、音量时长等也较为正常，说明该音乐播放器的设计基本符合要求。

通过本次实验，我加深了对 DDS 频率合成原理和自顶向下模块化设计方法的理解。从 DDS 子模块到整机联调，逐层仿真验证的过程让我体会到，在复杂数字系统设计中，合理的模块划分和充分的仿真验证是保证设计正确性的关键。此外，开关防颤动、亚稳态同步化等问题看似细节，却直接影响用户体验和系统可靠性，深刻认识到工程实践中“细节决定成败”的道理。总体而言，本实验将教材中离散的知识串联为一个完整的音频播放系统，对 FPGA 数字系统设计有了更系统的把握。

七、思考题

7.1 在实验中，为什么 next_button、play_pause_button 两个按键需要消颤动及同步化处理，而 reset 按键不需要消颤动及同步化处理？

play_pause_button 和 next_button 在系统正常运行过程中被按下，其按键信号需要被控制器精确地采样以触发相应操作（切换播放/暂停状态、切换乐曲）。若不经消颤动处理，一次按键的机械颤动可能产生多个脉冲，导致被控制器误判为多次按键；若不经同步化处理，异步输入的按键信号可能在时钟边沿附近发生跳变，引发亚稳态，导致控制器产生不可预测的错误行为。因此这两个按键必须经过消颤动和同步化处理。

而 reset 按键不需要消颤动和同步化，原因有二：其一，reset 通常是上电或手动复位时的操作，按下的持续时间远大于颤动周期（百毫秒级 vs. 十毫秒级），系统中没有任何采样窗口会恰好只捕捉到颤动期的瞬态；其二，即使颤动导致了多次 reset 触发，reset 的最终效果始终是将系统复位到相同的初始状态，中间过程的波动不会影响最终结果。此外，在实际 FPGA 设计中，reset 信号通常以异步方式直接作用于所有寄存器的异步复位/置位端，天然不依赖时钟边沿采样，因此同步化也非必需。

7.2 在主控制器 (mcu) 设计中，是否存在接收不到按键信息？若存在，概率多大？有没必要修改设计？

存在按键丢失的可能，但概率极低，不需要修改设计。

原因在于：mcu 的 FSM 在 PLAY 和 PAUSE 之外的过渡状态（IDLE_RESET、NEXT）只停留一个时钟周期。如果按键脉冲恰好与这些过渡状态在同一拍到达，按键信号不会被采样。IDLE_RESET 和 NEXT 各占 1 拍（100MHz 下为 10ns），PAUSE 和 PLAY 各占不定多拍（等待按键）。

具体计算：按键被漏掉的概率为：

$$P = \frac{\text{过渡状态时长}}{\text{一个完整操作周期}} = \frac{2 \times 10 \text{ ns}}{2 \times 10 \text{ ns} + t_{\text{按键间隔}}}$$

由于人类按键速度在数百毫秒量级（ $t_{\text{按键间隔}} \approx 100 \text{ ms}$ ），该概率约为：

$$P \approx \frac{20 \text{ ns}}{100 \text{ ms}} = 2 \times 10^{-7}$$

完全可以忽略不计。因此无需修改 FSM 设计。

Für Elise

致爱丽丝

EveryonePiano

♩ = 55

Piano

6

11

16

21

1. 2.

图 20: 附:《致爱丽丝》五线谱